



# Multi-Agent & Kết Nối Hệ Thống

*AICB-P1 · Ngày 9 · MCP, A2A & LangGraph*

---

**Trần Quang Thiện**

CTO @ TrustedAI

VinUniversity · Phase 1 · Tuần 2 · 2026

## HÃY SUY NGHĨ...

*“Bạn có 1 agent rất giỏi. Nhưng bài toán đã quá lớn cho 1 agent. Làm thế nào để hệ thống vẫn rõ vai trò, dễ kiểm soát, và dễ mở rộng?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Khi Một Agent Không Còn Đủ
2. Multi-Agent Patterns
3. Supervisor-Worker Pattern
4. MCP - Model Context Protocol
5. A2A - Agent to Agent Communication
6. Orchestration Với LangGraph
7. Trace & Độ Tin Cậy

1

## Khi nào một agent không còn đủ?

Dấu hiệu nào cho biết nên chia nhỏ task cho nhiều agent — và khi nào chia nhỏ chỉ làm hệ thống rối thêm

2

## Chia việc theo kiểu nào?

4 pattern phối hợp, mỗi pattern trị một loại phức tạp khác nhau

3

## Các agent nối với nhau ra sao?

MCP nối agent với tool ngoài, A2A nối agent với agent — khác nhau ở đâu

4

## Hỏng thì lần ra ở đâu?

Vì sao multi-agent khó debug, và một trace đọc được cần chứa những gì

## Cách học hôm nay

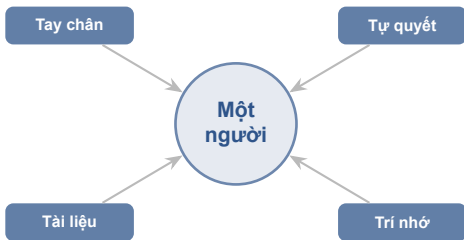
Mục tiêu là **hiểu để chọn đúng**: biết bài toán nào cần multi-agent, chọn pattern nào, và nhìn ra hệ thống đang hỏng ở đâu. Phần build nằm ở buổi hands-on riêng.

1. Khi Một Agent Không Còn Đủ
2. Multi-Agent Patterns
3. Supervisor-Worker Pattern
4. MCP - Model Context Protocol
5. A2A - Agent to Agent Communication
6. Orchestration Với LangGraph
7. Trace & Độ Tin Cậy

# Khi Một Agent Không Còn Đủ

---

Day 08 giúp agent biết tra tài liệu và trả lời có dẫn nguồn; Day 09 trả lời câu hỏi khi nào một agent không còn đủ để gánh toàn bộ bài toán



Mỗi buổi trước đều là **trang bị thêm cho cùng một người**: cho tay chân, cho quyền tự quyết, cho tài liệu để tra.

Một người giỏi tới mấy vẫn có trần: chỉ làm được một việc tại một thời điểm, và ôm càng nhiều vai thì càng dễ nhầm.

## Tool calling

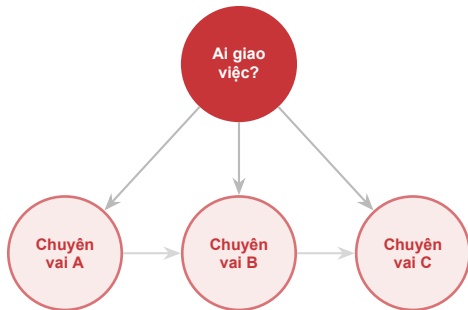
Gọi được hàm ngoài: tra đơn hàng, mở ticket, gọi API

## ReAct agent

Vòng lặp Think–Act–Observe: tự chọn tool, đọc kết quả, tự quyết bước tiếp theo

## RAG

Đọc tài liệu công ty và trả lời có dẫn nguồn, thay vì đoán theo trí nhớ huấn luyện



Lập đội thì câu hỏi đổi hẳn: **chia vai thế nào, ai giao việc cho ai, họ trao đổi ra sao, và hỏng thì biết tại khâu nào.**

Đó đúng là 4 câu hỏi của cả ngày hôm nay — và cũng là lý do một đội tôi thường thuê hơn một người

## Single agent

Một agent ôm mọi tool và mọi vai.  
Điểm ta đang đứng sau Day 08 — càng thêm tool thì prompt càng dài và càng khó kiểm soát

## Multi-agent

Nhiều agent, mỗi agent một vai hẹp, phối hợp để cùng ra một kết quả.  
Giao việc từ một agent điều phối chỉ là một trong nhiều kiểu phối hợp



## 1. Chật chỗ trong một lần suy luận

context bottleneck

Quá nhiều luật, kết quả tool và lịch sử chat chen vào cùng một lượt

## 2. Ôm nhiều vai thì không giỏi vai nào

specialization trade-off

Càng thêm vai, prompt càng dài và các luật càng dễ va nhau

## 3. Chạy nối đuôi, không chạy cùng lúc

parallelism

Nhiều việc chẳng liên quan gì nhau vẫn phải xếp hàng chờ

## 4. Lỗi không có ranh giới

Một bước sai làm bắn luôn context chung, và vì mọi vai nằm trong một prompt, không biết vai nào đã sai

### Đọc bản đồ này cho sòng phẳng

Ba slide sau mổ kỹ giới hạn 1–3. Riêng giới hạn 4: chia agent **không làm lỗi biến mất** — lỗi vẫn xảy ra, thậm chí nhiều chỗ hỏng hơn. Cái nó đổi được là mỗi lỗi có

System prompt của **một** agent CS, sau vài tháng thêm dần:



**240 dòng luật** mà model phải cân **cùng một lúc**, cho mọi câu hỏi — kể cả câu “shop mấy giờ đóng cửa?”.

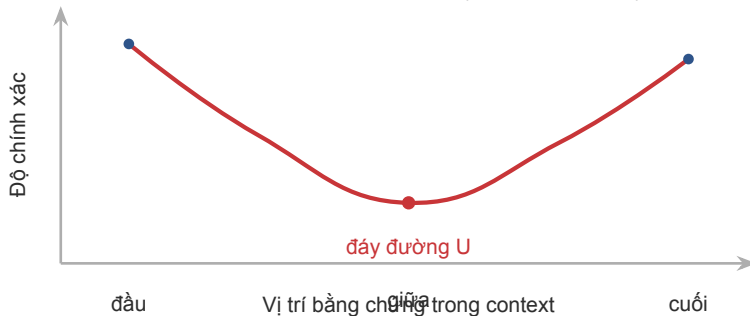
## Giới hạn 1 — context bottleneck:

Mỗi lượt, 240 dòng luật cộng lịch sử chat cộng kết quả tool đều nằm trong cùng một lần suy luận. Phần dành cho việc thật sự cần nghĩ bị ép lại.

## Giới hạn 2 — specialization trade-off:

Luật va nhau: “gặp lỗi thì retry” đấu với “không chắc thì hỏi lại người dùng”. Thêm tool thứ 9 phải sửa prompt, rồi nơm nớp lo bốn nhóm luật còn lại vỡ theo.

*Liu et al., Lost in the Middle, TACL 2024 (arXiv:2307.03172), Stanford*



**Lost in the middle** — Model dùng tốt thông tin ở đầu và cuối context, nhưng bỏ quên bằng chứng nằm ở giữa — kể cả khi nó thực sự có trong context. Nhồi thêm tài liệu vào một agent không làm nó suy luận tốt hơn.

Cùng một việc — đọc **40 PDF** rồi viết một báo cáo có trích dẫn — làm theo hai kiểu:

**40**

lượt **nối đuôi** nếu  
một agent tự đọc

**40**

agent đọc **cùng**  
**lúc** nếu chia ra

**4,5×**

chênh lệch  
thời gian

**1.500+**

tool call một  
lần giao việc

*Mức chênh 4,5× và 1.500+ tool call: Kimi Agent Swarm, 2026 — hệ thống này chạy tới 100 agent song song.*

*[kimi.com/blog/agent-swarm](https://kimi.com/blog/agent-swarm)*

**Một agent chỉ làm được một bước tại một thời điểm.** Đọc 40 PDF là 40 lượt nối đuôi nhau, dù chúng chẳng liên quan gì đến nhau và hoàn toàn có thể đọc cùng lúc. Tổng thời gian vì thế tăng theo **số bước**, không theo độ khó — và model mạnh hơn chỉ rút ngắn từng bước, không bỏ được hàng đợi.

## HOẠT ĐỘNG TRÊN LỚP

4 phút

Lấy **đề án cuối khoá** của bạn — hoặc bất cứ thứ gì bạn đang xây tới giờ — làm đối tượng.

1. **Viết một câu** mô tả nó làm gì cho người dùng.
2. **Liệt kê 4–6 việc** nó phải làm từ lúc nhận yêu cầu tới lúc trả kết quả.
3. **Khoanh những việc cần chuyên môn khác hẳn nhau** — ví dụ tìm tài liệu và gọi API là hai loại việc rất khác.

*Chưa cần quyết định gì cả. Giữ tờ này — cuối section ta sẽ quay lại nó với đủ tiêu chí để quyết. (4 phút cá nhân)*

## Deep research

Tìm rộng, rồi chia thành nhiều nhánh đào sâu song song, quay lại bổ sung nguồn khi còn mỏng

## Sản xuất nội dung

Manager giao research, drafting, editing cho 3 agent rồi duyệt bản gộp

## Customer service

1 agent đọc sentiment, 1 truy lịch sử đơn, 1 soạn trả lời — trong cùng luồng

## Security testing

Red-team dò lỗ hổng và chain exploit, blue-team phát hiện và vá từng lỗ

## Phân tích tài chính

Classifier nhanh quét bất thường, model lớn bình luận vĩ mô, agent thứ ba lấy giá live

## Review nhiều góc

VC hoài nghi + PM kỳ cựu + chuyên gia đạo đức cùng mổ một kế hoạch launch

## Điểm chung

Không phải bài toán khó hơn, mà là bài toán **quá rộng, quá dài, hoặc cần nhiều góc nhìn** cho một agent — và chia được thành các vai chạy song song hoặc soi lẫn nhau.

| Tiêu chí         | Single-agent                     | Multi-agent                                             |
|------------------|----------------------------------|---------------------------------------------------------|
| Chi phí LLM      | 1 lần gọi mỗi lượt               | mỗi agent $\geq 1$ lần gọi, supervisor là lần gọi riêng |
| Latency          | thấp, một mạch                   | cao hơn vì nhiều chặng nối tiếp                         |
| Khả năng debug   | khó: mọi thứ trộn trong 1 prompt | rõ: trace ghi riêng theo từng agent                     |
| Chuyên môn hoá   | giỏi đều, không sâu vai nào      | mỗi worker hẹp vai, sâu hơn                             |
| Khả năng mở rộng | thêm vai = prompt phình to       | thêm 1 worker mà không viết lại gốc                     |
| Độ phức tạp      | thấp, dễ bắt đầu                 | cao: state, route, contract phải quản                   |

*Ba dòng trên là cái mất, ba dòng dưới là cái được — checklist ở slide sau chính là cách cân hai bên*

- ✓ **Task có nhiều bước vai trò khác nhau:** plan, retrieve, tool use, tổng hợp
- ✓ **Có thể chia việc độc lập:** 2 worker làm song song vẫn hợp lý
- ✓ **Cần debug rõ ai làm sai:** route sai, retrieve sai, hay synthesis sai
- ✓ **Cần mở rộng dần:** thêm 1 worker mới mà không viết lại cả prompt gốc
- ☐ **Đừng chia khi mọi vai đều cần cùng một ngữ cảnh** — lúc đó chia chỉ tốn thêm công chuyển tin
- ☐ **Đừng chia chỉ vì thấy “ngẫu”.** Multi-agent tốn khoảng  $15\times$  token so với một lượt chat; nếu một agent đã đủ, giữ đơn giản sẽ rẻ và ổn định hơn

Số liệu: Anthropic, “How we built our multi-agent research system”, 2025.



## HOẠT ĐỘNG TRÊN LỚP

8 phút

Lấy lại tờ A0 — danh sách việc mà đề án của bạn phải làm.

1. **Soi từng tiêu chí** ở slide trước: có nhiều loại vai không · chia độc lập được không · cần biết rõ ai làm sai không · có phải mở rộng dần không.
2. **Đánh dấu** chỗ khó debug nhất hiện giờ — khi ra kết quả sai, bạn có biết hỏng ở khâu nào không?
3. **Kết luận một câu:** nên chia, hay giữ một agent. Nếu nên chia, chọn **một vai** chuyển ra trước tiên và viết lý do.

*“Giữ một agent” là kết luận hoàn toàn hợp lệ — nói được vì sao mới là điều cần. (8 phút nhóm + 2 phút chia sẻ)*

## CHECKPOINT — Tự kiểm tra

**Vì sao nhồi thêm tài liệu vào một agent không làm nó suy luận tốt hơn?**

✓ Context window có hạn và “lost in the middle”: thêm token đầy bằng chứng vào vùng giữa dễ bị model bỏ quên, đồng thời bào mòn phần token còn lại để suy luận.

**Nêu 2 trong 4 giới hạn cốt lõi của single-agent.**

✓ Chọn 2: chật chỗ trong một lần suy luận · ôm nhiều vai thì không giỏi vai nào · chạy nổi đuôi không chạy cùng lúc · lỗi không có ranh giới.

**Một agent đang có 8 tool. Thêm tool thứ 9 làm prompt dài thêm 40 dòng luật — đó là tín hiệu gì?**

✓ Tín hiệu quá tải của cả giới hạn 1 và 2: mỗi câu hỏi, kể cả câu đơn giản nhất, đều phải cân thêm 40 dòng luật; và nhóm luật mới có thể va với luật cũ. Đây là lúc cân nhắc chuyển nhóm việc đó thành một worker riêng.

*Trả lời ngắn gọn, bằng lời của bạn.*

1. Khi Một Agent Không Còn Đủ
- 2. Multi-Agent Patterns**
3. Supervisor-Worker Pattern
4. MCP - Model Context Protocol
5. A2A - Agent to Agent Communication
6. Orchestration Với LangGraph
7. Trace & Độ Tin Cậy

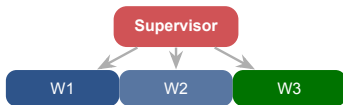
# Multi-Agent Patterns

---

Có nhiều cách chia hệ thống thành nhiều agent; điều quan trọng là chọn pattern giúp giải quyết đúng loại phức tạp, không tạo thêm phức tạp giả

02

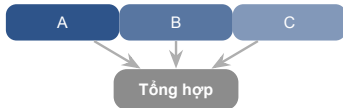
Supervisor-Worker



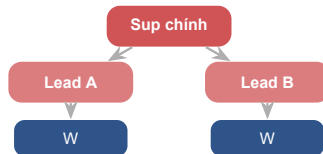
Pipeline



Debate



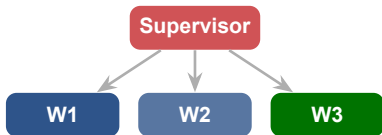
Hierarchical



### Đọc hình theo hình dạng

Supervisor-worker **toả ra** · Pipeline **nối tiếp** · Debate **hội tụ** · Hierarchical **lồng tầng** (supervisor của các supervisor). Bốn slide sau đi vào từng cái: dùng khi nào, ví dụ thật, và cái bẫy đi kèm.

# Pattern 1 — Supervisor-Worker: Một Agent Giao Việc, Nhiều Agent Làm



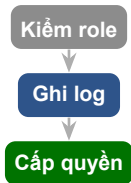
Mỗi worker nhận **một phần việc khác nhau**, ghép lại mới thành câu trả lời. Worker không nói chuyện với nhau. Trường hợp gọn nhất: supervisor chỉ làm **định tuyến** — đọc ticket rồi chọn đúng một worker (refund / kỹ thuật / escalate).

**Dùng** khi yêu cầu đa dạng, khó đoán trước, và các nhánh việc **độc lập** nhau.

**Ví dụ thật.** Deep research của Anthropic: một lead agent chạy **3–5 subagent song song**, mỗi subagent tìm một hướng riêng — rút ngắn tới **90%** thời gian với câu hỏi cần quét rộng.

**Bẫy 1 — giao việc mơ hồ.** Giao cho 3 worker cùng một câu “tìm hiểu về đối thủ X” thì cả 3 sẽ mở đúng một trang chủ và trả về gần như một bản. Phải viết rõ cho từng worker: **làm gì, trả về dạng gì, được dùng tool nào, và phần nào của người khác thì đừng đụng** — worker 1 lấy bảng giá, worker 2 lấy đánh giá người dùng, worker 3 lấy tin tuyển dụng.

**Bẫy 2 — supervisor ôm quá nhiều.** Nếu supervisor vừa route vừa tự làm, nó thành nút cổ chai và context của nó lại phình đúng như một agent đơn lẻ.



Các bước **cố định**, luôn chạy theo đúng thứ tự đó. Ví dụ: quy trình duyệt cấp quyền truy cập.

**Dừng khi** các bước đã cố định sẵn, gần như không rẽ nhánh.

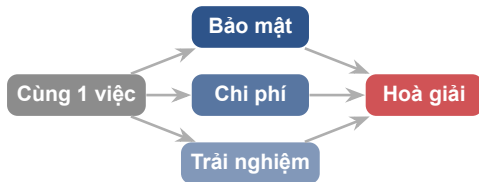
**Được gì.** Mỗi bước test riêng được, lỗi nằm ở bước nào thấy ngay.

**Bẫy 1 — thời gian cộng dồn.** Chuỗi chạy nối đuôi nên latency là tổng của mọi chặng; retry một bước có thể kéo chạy lại cả khúc sau.

**Bẫy 2 — gây khi yêu cầu đi chệch.** Pipeline không có chỗ để rẽ. Yêu cầu lạ vẫn bị ép qua đúng chuỗi đó.

**Mẹo chuyên tin.** Đừng nhét nguyên output bước trước vào prompt bước sau. Lưu ra ngoài rồi **chuyên một tham chiếu ngắn** (id, đường dẫn file) — cách Anthropic dùng để context không phình theo chiều dài chuỗi.

# Pattern 3 — Debate: Ba Agent Cùng Nhận Một Việc, Khác Nhau Ở Góc Nhìn



Ba agent đều đọc **đúng một input** và đều trả lời **trọn vẹn câu hỏi** — chỉ khác nhau ở vai được giao.

## Khác supervisor-worker chỗ nào.

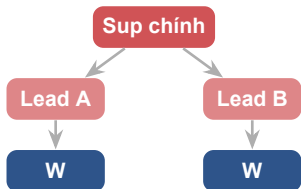
Supervisor-worker chia **khối lượng**: mỗi worker một mảnh, thiếu một mảnh là kết quả không đủ. Debate chia **góc nhìn**: bỏ đi một góc thì câu trả lời vẫn chạy, chỉ là dễ sót rủi ro hơn. Vì vậy bước cuối cũng khác — một bên **ghép mảnh**, một bên **xử lý ý kiến trái nhau**.

**Dùng khi** sai một lần thì đắt, và cái đáng sợ là **điểm mù** chứ không phải khối lượng việc.

**Bắt 1 — giá nhân lên.** Ba góc nhìn là ba lần token cho cùng một việc.

**Bắt 2 — chỗ khó nằm ở bước cuối.** Ba bên mâu thuẫn thì ai quyết? Phải chốt trước: lấy đa số, ưu tiên góc bảo mật, hay đẩy cho người duyệt.





Chính là supervisor-worker **lồng thêm một tầng**, khi số worker đã quá nhiều cho một người giao việc.

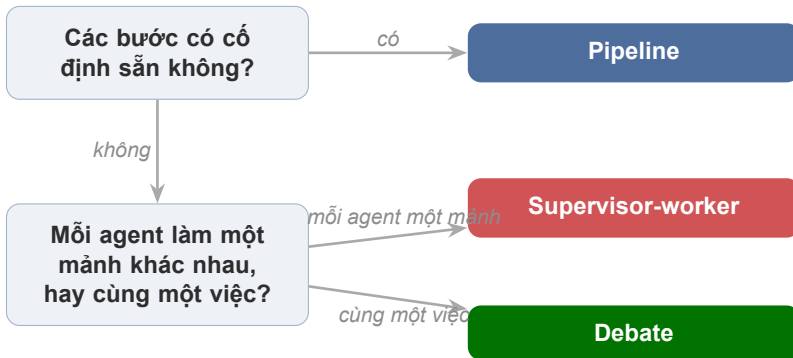
**Dùng khi** có nhiều nhóm việc khác hẳn nhau, mỗi nhóm cần luật giao việc riêng.

**Dấu hiệu cần thêm tầng.** Supervisor phải nhớ quá nhiều worker và quá nhiều luật route — đúng vấn đề context của Section 1, chỉ là đã lùi lên một tầng.

**Bẫy 1 — tam sao thất bản.** Yêu cầu đi qua hai lần diễn giải mới tới worker; mỗi lần là một cơ hội lệch ý.

**Bẫy 2 — trace dài.** Một lỗi giờ phải lần qua ba tầng mới ra thủ phạm.

*Đây là pattern nên để dành. Hầu hết bài toán trong lớp dừng ở supervisor-worker một tầng.*



## Còn Hierarchical?

Hierarchical **không phải một lựa chọn ngang hàng** ở đây — không ai mở đầu bằng nó. Nó là thứ bạn thêm vào **sau**, khi supervisor đã nhớ không xúi sổ worker và số luật giao việc.

## Tra một dữ kiện

1 agent, 3–10 lượt gọi tool. Chia agent ở đây chỉ tốn thêm

## So sánh vài lựa chọn

2–4 worker, mỗi worker 10–15 lượt gọi tool

## Quét rộng, nhiều hướng

10+ worker, mỗi worker một phần việc riêng, ranh giới rõ

## Vì sao phải ghi rõ con số

Nếu không ghi mức này vào prompt của supervisor, nó sẽ **gọi thừa worker cho việc nhỏ**. Con số lấy từ hệ thống deep research của Anthropic — không phải bắt buộc, nhưng là điểm khởi đầu tốt hơn là đoán.

*Nguồn: Anthropic, “How we built our multi-agent research system”, 2025.*

## HOẠT ĐỘNG TRÊN LỚP

10 phút

Lấy lại tờ A1. Lần này không chọn giúp người khác — chọn cho chính hệ thống bạn đang xây.

### Phần 1 — pattern bạn sẽ dùng (6 phút)

- Đi qua **hai câu hỏi** ở slide trước để chọn 1 pattern.
- Vẽ ra: có **những agent nào**, mỗi agent **nhận gì** và **trả về gì**, **ai ghép** kết quả cuối.
- Viết thêm **cái bẫy** của pattern đó sẽ rơi vào chỗ nào trong hệ thống của bạn.

### Phần 2 — một pattern khác (4 phút)

- Chọn thêm **1 pattern bạn KHÔNG dùng**, và mô tả **tình huống nào** sẽ khiến bạn cần tới nó — yêu cầu đối thể nào, quy mô lớn ra sao.

*Nếu ở A1 bạn kết luận “giữ một agent”: vẫn làm Phần 1 — chỉ ra bước nào sẽ là worker đầu tiên nếu sau này buộc phải chia.*

## CHECKPOINT — Tự kiểm tra

**Một trợ lý nội bộ nhận đủ loại yêu cầu khó đoán trước. Pipeline hay supervisor-worker, vì sao?**

✓ Supervisor-worker — câu hỏi đầu tiên (“các bước có cố định sẵn không?”) trả lời là không, nên pipeline loại ngay; cần một agent đọc yêu cầu rồi chọn đúng worker.

**Một team chọn debate pattern cho quy trình duyệt cấp quyền vốn đã cố định từng bước. Sai ở đâu, đổi sang gì?**

✓ Sai: quy trình đã cố định thì không cần nhiều góc nhìn — debate chỉ nhân chi phí lên và đề thêm việc hoà giải. Đổi sang pipeline (kiểm role → ghi log → cấp quyền).

**Điểm yếu cố hữu của pipeline khi yêu cầu đi chệch các bước cố định là gì?**

✓ Thời gian cộng dồn qua từng chặng, không có chỗ rẽ nhánh, và retry một bước có thể kéo chạy lại cả khúc sau.

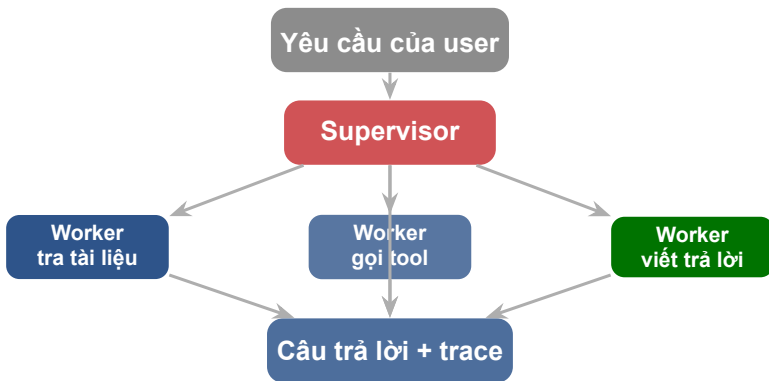
*Đáp án chỉ để tự đối chiếu — hãy thử trả lời trước khi mở.*

1. Khi Một Agent Không Còn Đủ
2. Multi-Agent Patterns
- 3. Supervisor-Worker Pattern**
4. MCP - Model Context Protocol
5. A2A - Agent to Agent Communication
6. Orchestration Với LangGraph
7. Trace & Độ Tin Cậy

# Supervisor-Worker Pattern

---

Pattern 1 đã biết mặt ở phần trước — giờ đi vào ba thứ quyết định nó chạy được hay không: vai trò, state, và phiếu giao việc



## Khung nghĩ đúng

Với một trợ lý nội bộ, supervisor **không** cần tự thuộc chính sách nghỉ phép hay quy trình xin cấp quyền. Việc của nó là **giao đúng việc, gọi đúng worker, và gom đầu ra thành một câu trả lời mạch lạc**.



## Supervisor

- đọc yêu cầu ban đầu, hiểu nó thuộc loại gì
- quyết định worker nào nên tham gia
- theo dõi việc đang chạy tới đâu, gọi lại nếu hỏng
- ghép các đầu ra thành câu trả lời cuối

## Worker

- làm **một** việc hẹp
- nhận đầu vào rõ, trả đầu ra rõ
- càng ít phụ thuộc lần gọi trước càng dễ test
- hỏng một worker không kéo sập cả hệ thống

## Ranh giới dễ nhớ

Supervisor giữ **luồng quyết định** decision flow; worker giữ **chuyên môn** domain skill. Dấu hiệu hỏng: một worker vừa làm việc hẹp của nó, vừa âm thầm gọi worker khác — lúc đó bạn có hai supervisor mà chỉ một cái được ghi trong sơ đồ.

```
SUPERVISOR_PROMPT = """
You are the Coordinator. ONLY task: classify the request
and call the right worker. Do NOT answer from your own
knowledge.

If info is missing to classify -> ask the user one question.

Worker list:
- RetrievalWorker: look up internal documents
- ToolWorker:      call systems (HR, ticketing, permissions)
- SynthesisWorker: write the final answer from evidence

Return ONLY the worker name. No explanation, no making up.
"""
```

**Lưu ý:** Quá nhiều tự do thì supervisor tự trả lời, bỏ qua worker, và bịa dù worker đã trả về bằng chứng. Quá ít thì nó đẩy mọi thứ sang một worker an toàn nhất và hệ thống thành một agent đơn lẻ đội lốt. Cách chỉnh: mô tả mỗi worker **thật ngắn**, ranh

## Hẹp specialized

Một worker một việc chính: tra tài liệu, gọi tool, tóm tắt, kiểm tra chính sách.

**Ví dụ.** “Worker tra tài liệu” chỉ tra — không kiêm luôn việc gọi hệ thống HR và viết câu trả lời.

## Độc lập từng lần gọi stateless

Worker làm được việc chỉ với đầu vào của lần gọi này — không cần nhớ những lần trước.

**Ví dụ.** Supervisor gửi “chính sách nghỉ phép 2025”, thay vì để worker tự đọc lại hội thoại đoán “cái đó” là cái gì.

## Test được testable

Đầu vào / đầu ra rõ để chạy thử riêng trước khi cắm vào supervisor.

**Ví dụ.** Worker tra tài liệu đã hứa trả **3 đoạn** mỗi câu hỏi: ném 20 câu mẫu vào, đếm số lần đoạn đúng có trong đó.

## Được gì khi định nghĩa worker rõ

Lúc hệ thống trả lời sai, bạn **cô lập được ngay**: chạy riêng đúng worker đó với đúng đầu vào của nó là biết nó hỏng hay không. Ba tính chất trên có mặt ở đây chính vì thế — hẹp để biết nhìn vào đâu, độc lập để chạy lại được một mình, test được để so đầu ra với kết quả mong đợi.

Đã làm gì  
history

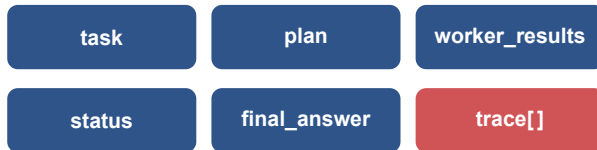
Còn đang dở  
pending

Vì sao chọn worker này  
decision

## Một mẫu state thật — đủ cả ba cột

```
{status:"in_progress",  
  history:["retrieval_worker: 3 đoạn từ hr_policy_2025.pdf"],  
  pending:["synthesis_worker"],  
  decision:"câu hỏi chính sách → tra tài liệu trước, chưa cần gọi tool"}
```

Chính status cho supervisor biết **khi nào dừng**: xong (done) hoặc hỏng (error).



**Lưu ý:** Thiếu `trace` thì chạy xong không ai biết **đường nào** dẫn tới kết quả — lỗi sinh ra ở worker tra tài liệu nhưng chỉ lộ ra ở bước viết trả lời, mà bạn không lần ngược được.

```
from typing import TypedDict, Optional

class Day09State(TypedDict):      # typed dict = clear schema
    task: str
    user_context: dict
    plan: list[str]
    retrieval_result: list[dict]
    tool_result: dict
    synthesis_draft: str
    final_answer: str
    status: str                  # pending|running|done|error
    need_retrieval: bool         # supervisor routing flag
    need_verify: bool           # supervisor routing flag
    trace: list[dict]           # append-only, 7-field entries
    error: Optional[dict]       # {code, reason} or None
```

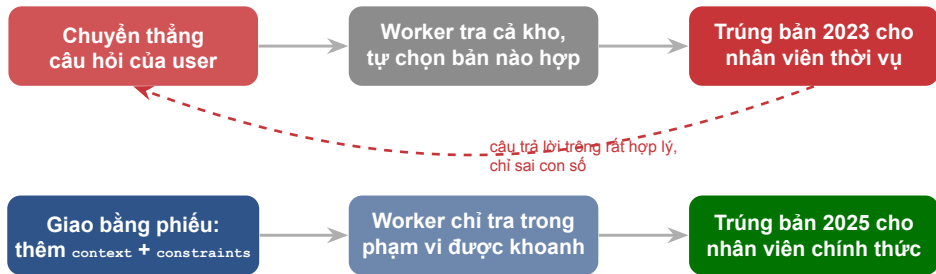
*Một state duy nhất cho cả hệ thống. `trace` chỉ ghi thêm, không ghi đè · `error` là `{code, reason}` để hông có kiểm soát · `need_retrieval/need_tool` là cờ do supervisor ghi để định tuyến · mỗi worker chỉ ghi vào field của mình. Đây đúng là state bạn dùng ở buổi hands-on.*

**Phiếu giao việc** (contract) — bản mô tả cố định cho *một lần* supervisor giao việc cho worker: phải gửi kèm những gì, và worker bắt buộc trả về dạng gì. Hai bên thoả thuận trước, không đoán lúc chạy.

```
{  
  "task": "tra_tai_lieu",  
  "context": "chính sách nghỉ phép, áp dụng từ 2025",  
  "constraints": ["chỉ tài liệu nội bộ"],  
  "expected_output": "3 đoạn liên quan nhất",  
  "error": { "code": null, "reason": null }  
}
```

Năm field này đủ để worker **test riêng được** (biết đầu vào đầu ra), **ghi được vào trace**, và **thay được** bằng worker khác mà supervisor không phải sửa.

User hỏi: “nghỉ thai sản được bao lâu?” — kho tài liệu có nhiều bản, nhiều năm, nhiều loại hợp đồng.



**Lưu ý:** Câu hỏi của user **nghe thì đủ**, nhưng thiếu đúng hai thứ chỉ supervisor mới biết: **người hỏi thuộc diện nào** và **bản nào đang hiệu lực**. Không truyền xuống thì worker phải tự chọn — và nó chọn im lặng, không báo lỗi. Đó là lý do `context` và `constraints` nằm trong phiếu.



viết ba kiểu:

## Quá ít

"nghĩ thai sản được bao lâu?"

Nguyên văn câu user — worker phải tự chọn bản nào, như slide trước.

## Vừa đủ

"nghĩ thai sản, nhân viên chính thức, 2025"

Đủ để worker khỏi đoán, không thừa một chữ.

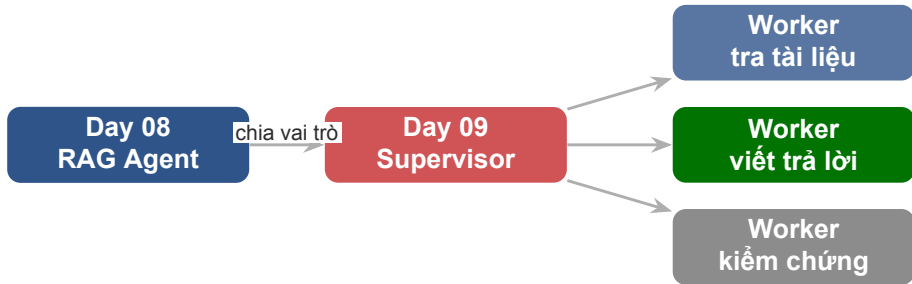
## Quá nhiều

"...12 lượt hội thoại + hồ sơ nhân viên + sổ tay 40 trang"

Tốn token, và worker dễ bám vào chi tiết không liên quan.

## Cách làm

Bắt đầu từ cột giữa: chỉ ghi thứ worker **cần để làm xong việc**. Thiếu thì thêm — worker trả sai là biết ngay thiếu gì. Thừa thì khó cắt: sau vài tháng không ai dám bỏ field nào vì không rõ worker nào còn đọc nó.



## Thông điệp lab

Day 09 không bắt đầu từ số 0. Ta lấy đúng năng lực tra tài liệu và trả lời của Day 08, **tách thành hai worker**, rồi thêm một worker Day 08 chưa có: **kiểm chứng** — đối chiếu từng câu trong bản trả lời với đoạn tài liệu đã lấy, câu nào không có nguồn thì bỏ. Đây chính là thứ một agent đơn lẻ khó tự làm: nó không giỏi tự bắt lỗi của chính mình.

## HOẠT ĐỘNG TRÊN LỚP

8 phút

Lấy lại bản vẽ ở **A2 Phần 1**. Ở đó bạn mới có các ô và mũi tên. Bây giờ điền vào một ô.

### Phần 1 — một phiếu giao việc (5 phút)

- Chọn **một worker** trong bản vẽ của bạn — chọn cái bạn thấy mơ hồ nhất.
- Viết đủ **5 field**: `task`, `context`, `constraints`, `expected_output`, `error`.
- `context` phải là câu cụ thể, không phải “thông tin liên quan”.

### Phần 2 — worker nào ghi field nào (3 phút)

- Liệt kê các field **state** hệ thống bạn cần, rồi ghi bên cạnh mỗi field **ai là người ghi nó**.
- Nếu một field có **hai người ghi** — khoanh tròn. Đó là chỗ sẽ hỏng đầu tiên.

**Giữ tờ này.** Phiếu ở Phần 1 là thứ bạn gõ thành code ở buổi hands-on; bảng ở Phần 2 chính là *Day09State* của bạn.

## Phần 1. Phiếu cho **worker** tra tài liệu:

```
task: "tra_tai_lieu"  
context: "nghỉ thai sản,  
        nhân viên chính thức, 2025"  
constraints: ["tài liệu nội bộ",  
             "bản đang hiệu lực"]  
expected_output: "3 đoạn +  
                tên file"  
error: {code, reason}
```

Hai dòng *context* và *constraints* chính là hai thứ đã thiếu ở slide “chuyển thẳng câu hỏi của user”.

## Phần 2. Field nào, ai ghi:

|                     |                           |
|---------------------|---------------------------|
| task, plan          | supervisor                |
| status              | supervisor                |
| retrieval_result    | worker tra tài liệu       |
| draft_answer        | worker viết trả lời       |
| <b>final_answer</b> | <b>hai người cùng ghi</b> |
| trace[]             | mọi worker (chỉ thêm)     |

**Chỗ khoan tròn.** Worker viết trả lời và worker kiểm chứng đều muốn ghi `final_answer`. Chạy xong không ai biết bản cuối là bản nào. Cách sửa: worker viết trả lời ghi `draft_answer`, chỉ worker kiểm chứng được ghi `final_answer`.

**Một field một chủ** — đó là thứ Phần 2 dùng để tìm ra.

## CHECKPOINT — Tự kiểm tra

### Supervisor có cần giỏi hơn mọi worker không? Vì sao?

✓ Không. Việc của nó là giao đúng việc, gọi đúng worker và gom kết quả — không phải tự làm phần chuyên môn. Supervisor tự trả lời là dấu hiệu ranh giới viết chưa đủ rõ.

### Kể tên 3 trong 5 field của một phiếu giao việc, và mỗi field chặn được cái gì.

✓ `task/context` chặn worker đoán mò · `constraints` chặn nó lấy nguồn không được phép · `expected_output` chặn trả về sai dạng · `error` để hồng có kiểm soát thay vì im lặng.

### Một state không có field `trace`: chính xác việc gì không còn lần ngược được?

✓ Thứ tự worker đã chạy và đầu vào/đầu ra ở mỗi bước. Lỗi lộ ra ở bước viết trả lời nhưng không truy về được điểm gốc (ví dụ lúc giao việc cho worker tra tài liệu đã thiếu ngữ cảnh).

*Nếu chưa chỉ ra được điều gì không lần ngược được ở câu 3, xem lại slide “`trace` là bắt buộc”.*

# GIẢI LAO

---

Giải lao 10 phút



1. Khi Một Agent Không Còn Đủ
2. Multi-Agent Patterns
3. Supervisor-Worker Pattern
- 4. MCP - Model Context Protocol**
5. A2A - Agent to Agent Communication
6. Orchestration Với LangGraph
7. Trace & Độ Tin Cậy

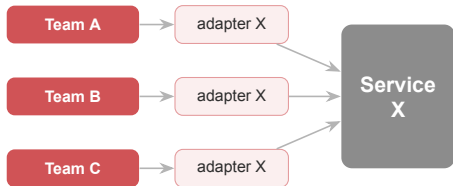
# MCP - Model Context Protocol

---

Mỗi team tự viết adapter cho cùng một service là lãng phí và dễ vỡ — MCP để bên sở hữu service công bố năng lực một lần, theo một chuẩn, cho mọi agent

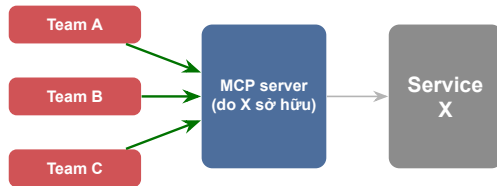


## Mỗi bên tự lo



*Ba team viết ba lần cùng một thứ.  
X đổi API → cả ba phải sửa,  
ai quên thì hỏng lúc chạy*

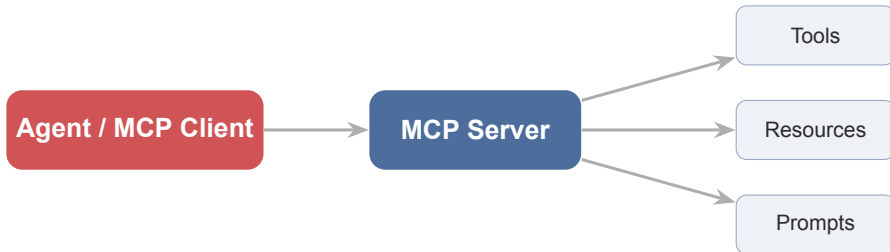
## X tự công bố theo chuẩn



*X viết **một** lần cho mọi agent.  
X đổi API → sửa trong server,  
không team nào phải sửa*

## Lợi cả hai chiều

Đây không phải chuyện tiết kiệm code trong một repo, mà là chuyện **ai chịu trách nhiệm**. **Bên có service** công bố năng lực đúng một lần, theo một chuẩn — và tiếp cận được mọi agent. **Bên làm agent** học đúng một chuẩn — và dùng được mọi service đã theo chuẩn. Trách nhiệm về đúng người biết rõ service nhất.



## Ổ cắm chuẩn

Giống **cổng USB** — agent học *một* chuẩn, cắm được vào mọi tool theo chuẩn đó. Worker tra tài liệu không cần biết kho tài liệu chạy trên hạ tầng nào hay đăng nhập kiểu gì; nó chỉ đọc surface mà server công bố. Anthropic mở MCP ngày **25/11/2024**, mã nguồn mở — nên server của bên khác viết ra bạn cũng cắm được.

## Resources

Là gì: dữ liệu agent **đọc** được.

Ví dụ: file chính sách nghỉ phép 2025.

## Tools

Là gì: hành động agent **gọi** được.

Ví dụ: tạo một đơn nghỉ phép trong hệ thống HR.

## Prompts

Là gì: mẫu câu lệnh server **gợi ý** sẵn.

Ví dụ: mẫu để trả lời câu hỏi chính sách.

## Khác nhau ở chỗ nào

**Resources chỉ đọc:** gọi lại bao nhiêu lần cũng không ảnh hưởng gì. **Tools thay đổi dữ liệu thật:** gọi sai một lần là hệ thống HR phát sinh một đơn nghỉ phép mà nhân viên không hề tạo. Vì vậy Tools phải giới hạn quyền chặt hơn, và với tool rủi ro cao thì cần người xác nhận trước khi chạy.



## Vì sao quan trọng

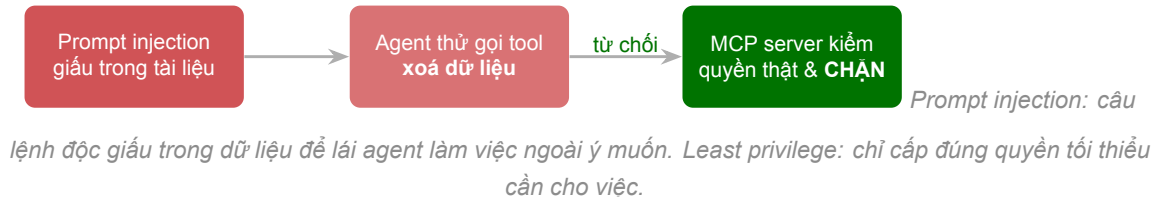
Agent **không** được lập trình sẵn để biết tool `search_policy` tồn tại. Nó phát hiện tool lúc chạy — thêm tool mới vào server là agent dùng được ngay, không sửa code.

# Ví Dụ Thật: Server Tài Liệu Mở `search_policy` Cho Worker Tra Tài Liệu

```
# retrieval worker calls the tool via the MCP surface
result = mcp.call("search_policy", {
    "query": "nghỉ thai sản",
    "employee_type": "chính_thuc",
    "effective_year": 2025,
})
# returned JSON (trimmed):
# {
#   "chunks": [
#     {"text": "Nghỉ thai sản 6 tháng...",
#      "source": "hr_policy_2025.pdf#p12"}
#   ],
#   "count": 3
# }
```

## Vì sao đáng giá

Đội HR đổi tài liệu thì chỉ cập nhật **MCP server của họ**; supervisor và các worker



## Ba lớp phòng thủ

- 1) Chỉ cấp quyền tối thiểu (read-ticket chứ không delete-DB).
- 2) Hành động nguy hiểm (hoàn tiền) phải có **người duyệt**.
- 3) Chạy server trong **container cô lập** (sandbox).

1. Khi Một Agent Không Còn Đủ
2. Multi-Agent Patterns
3. Supervisor-Worker Pattern
4. MCP - Model Context Protocol
- 5. A2A - Agent to Agent Communication**
6. Orchestration Với LangGraph
7. Trace & Độ Tin Cậy

# A2A - Agent to Agent Communication

---

MCP giúp agent nói chuyện với tool; A2A giúp agent nói chuyện với agent khác theo cách rõ nhiệm vụ, rõ bối cảnh, và rõ đầu ra mong đợi

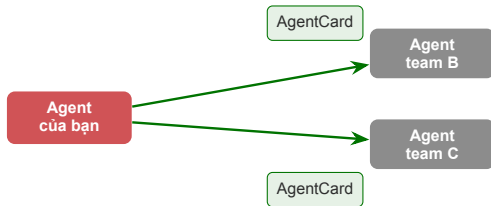


## Chưa có chuẩn: mỗi cặp tự thoả thuận



Mỗi cặp tự bịa: gọi ở đâu, gửi dạng gì, báo tiến độ kiểu nào, từ chối thì trả về gì

## Có A2A: mỗi agent tự công bố năng lực



Bên kia công bố **một lần**: làm được gì, gọi ra sao — ai theo chuẩn cũng giao việc được

## Vì sao không coi nó như một tool cho xong

Ba thứ khiến agent khác hẳn tool. Nó **tự quyết**: có thể hỏi lại, hoặc làm theo cách bạn không lường trước. Nó **làm lâu**: một việc chạy 20 phút thì không trả lời gọn trong một lần gọi. Và nó **có quyền từ chối**: hết quota, ngoài phạm vi, hoặc không đủ thẩm quyền. Ba thứ đó phải nằm trong cách giao tiếp ngay từ đầu.

## MCP

- agent nói chuyện với **tool / capability**
- mục tiêu là **kết nối năng lực bên ngoài**
- tool chỉ **thực thi**, không tự quyết định

## A2A

- agent nói chuyện với **agent khác**
- mục tiêu là **chia việc và đồng bộ**
- agent kia **tự quyết**: có thể hỏi lại, từ chối, hoặc phản biện

## Một câu để nhớ

**MCP** trả lời “**lấy năng lực ở đâu**”; **A2A** trả lời “**giao việc cho ai**”. Thử bằng cách hỏi: gọi xong thì bên kia có thể **làm khác ý bạn** không? Tool thì không — gọi là chạy đúng thể. Agent thì có — nó hỏi lại, chạy 20 phút rồi trả từng phần, hoặc từ chối thẳng.

## Vẫn là phiếu giao việc của Section 3

```
task · context · constraints  
expected_output · error
```

Trong nội bộ một hệ, chừng đó là đủ: bạn sở hữu cả hai đầu, sai thì sửa cả hai.

Qua ranh giới thì không — bạn **không sửa được** agent bên kia, và nó **không chờ bạn**.

## Bốn thứ phải thêm

|            |                                            |
|------------|--------------------------------------------|
| from       | ai gửi — để bên kia biết có nên nhận không |
| deadline   | hạn chót — quá hạn thì bạn thôi chờ        |
| status_url | báo tiến độ ở đâu, vì việc chạy lâu        |
| on_reject  | từ chối thì trả về gì, và bạn làm gì tiếp  |

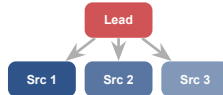
**AgentCard.** Để biết bên kia *làm được gì* trước khi giao, mỗi agent công bố một thẻ mô tả: kỹ năng nào, gọi ở đâu, xác thực kiểu gì. Đúng mạch MCP — bên có năng lực công bố một lần, cho mọi người dùng.

## Handover — tuần tự

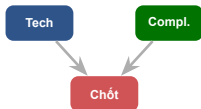


A xong việc thì giao cho B

## Cooperative Search — song song



## Joint Decision — đồng thuận

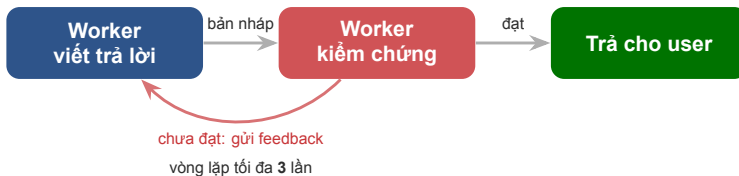


## Self-Correction — viết – kiểm – sửa



## Ba cái đầu bạn đã gặp rồi

Handover là **pipeline**, Cooperative Search là **supervisor-worker**, Joint Decision là **debate** — vẫn bốn pattern của Section 2, chỉ khác ở chỗ bên kia **không phải agent của bạn**. Chỉ Self-Correction là mới: một agent viết, một agent khác soi và trả về để



## Vì sao phải chặn số vòng

Worker kiểm chứng soi từng câu, thiếu nguồn thì trả về bắt viết lại. Không chặn thì hai worker đẩy qua đẩy lại mãi — mỗi vòng là thêm một lần gọi model, tốn tiền mà không chắc tốt hơn. Quá **3 vòng** thì dừng, trả phần chắc chắn và báo chỗ chưa có nguồn.

1. Judge agent — trọng tài đọc bằng chứng hai bên rồi phân xử

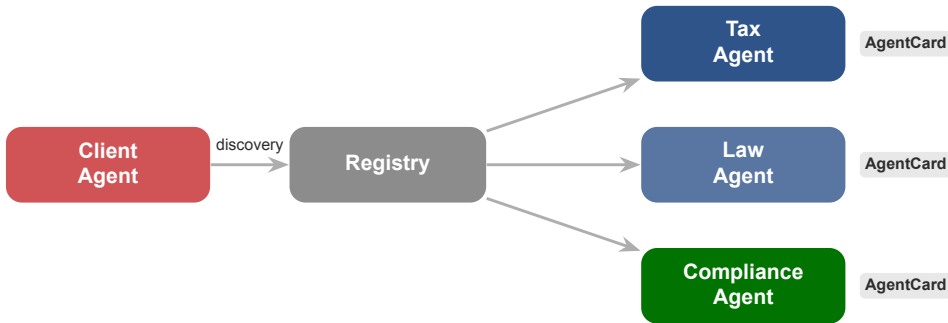
2. Hierarchy — một vai được ưu tiên cứng theo luật

Hierarchy:  
Compliance  
luôn thắng  
Support

3. Con người — last resort khi máy không tự khép được

## Xung đột cụ thể

Agent A nói **được hoàn tiền**; agent B nói khách **vi phạm chính sách bảo mật nên không**. Judge cân bằng chứng; nếu vẫn kẹt, Hierarchy cho Compliance thắng; còn tranh cãi về ngoại lệ thì đẩy con người.



## AgentCard là gì

Một tài liệu JSON metadata mô tả skills, auth, endpoint; thường phục vụ tại `/.well-known/agent-card.json`. Client khám phá agent **lúc runtime** qua Registry — không hard-code địa chỉ từng agent như trong một codebase.

- ☒ **Biết rõ ai được gọi ai:** không phải agent nào cũng được chạm mọi capability
- ☒ **Biết dữ liệu nào được truyền đi:** tránh đẩy thừa PII hoặc state nhạy cảm
- ☒ **Biết output nào cần xác thực lại:** đặc biệt khi worker chạm tool ngoài
- ☐ **Đừng giả định mọi agent đều đáng tin như nhau.** Hệ thống nhiều agent vẫn cần trust boundary



# GIẢI LAO

---

Giải lao 10 phút



1. Khi Một Agent Không Còn Đủ
2. Multi-Agent Patterns
3. Supervisor-Worker Pattern
4. MCP - Model Context Protocol
5. A2A - Agent to Agent Communication
- 6. Orchestration Với LangGraph**
7. Trace & Độ Tin Cậy

# Orchestration Với LangGraph

---

Sau khi hiểu vai trò và giao tiếp, ta cần một cách biểu diễn luồng chạy rõ ràng; LangGraph biến điều phối ẩn-trong-prompt thành graph tường minh

| Framework         | Cách mô tả hệ                                         | Hợp nhất với                                             | Điểm mạnh                                                |
|-------------------|-------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------|
| CrewAI            | khai báo <b>vai</b> và nhiệm vụ, có cấp trên—cấp dưới | quy trình nội dung, nghiên cứu — đội nhỏ tới vừa         | giao việc theo vai, cả nhóm cùng chạy một “crew”         |
| AutoGen           | <b>hội thoại</b> nhiều lượt giữa các agent            | viết code, agent phản biện, giải dần từng vòng — đội vừa | lập trình bằng hội thoại, chat nhiều agent               |
| <b>LangGraph</b>  | <b>graph</b> : node, edge, state dùng chung           | workflow lớn: rẽ nhánh, lặp, lưu lại được                | state graph sẵn có, gắn thẳng vào hệ LangChain           |
| OpenAI Agents SDK | <b>nhẹ</b> : agent này chuyển việc cho agent kia      | làm thử nhanh, dự án nhỏ                                 | ít code khai báo, có sẵn trace và định tuyến chuyển việc |

*Cả bốn đều dựng được supervisor-worker; khác nhau ở cách chúng bắt bạn mô tả hệ thống. Bài này dùng LangGraph vì cần rẽ nhánh, chen người duyệt, và trace từng bước*

## Không framework

- routing chôn trong một prompt điều phối dài
- không biết hệ thống đang ở bước nào
- thêm một nhánh xử lý phải sửa cả prompt
- debug bằng cách rải `print`

## LangGraph

- routing thành code đọc được, review được
- graph vẽ ra được: thấy state đi qua đâu
- state có schema (**TypedDict**)
- có điểm dừng rõ ràng để **con người duyệt** human-in-the-loop

- **Node** là một hàm Python nhận `state` và trả về `state` mới. Ví dụ: `retrieval_worker(state)` đọc câu hỏi, ghi 3 đoạn tra được vào `state`.
- **Edge thường** nối cứng  $A \rightarrow B$ : chạy xong A luôn sang B.
- **Edge có điều kiện** là một hàm trả về **tên node kế tiếp** dựa trên `state`. Ví dụ: `supervisor` đọc câu hỏi nghĩ thai sản, trả về `"retrieval_worker"`.
- **State** là một `TypedDict`/`Pydantic` đi xuyên suốt graph: mỗi node **đọc toàn bộ**, nhưng chỉ **ghi field được phép**.

## Vì sao quan trọng

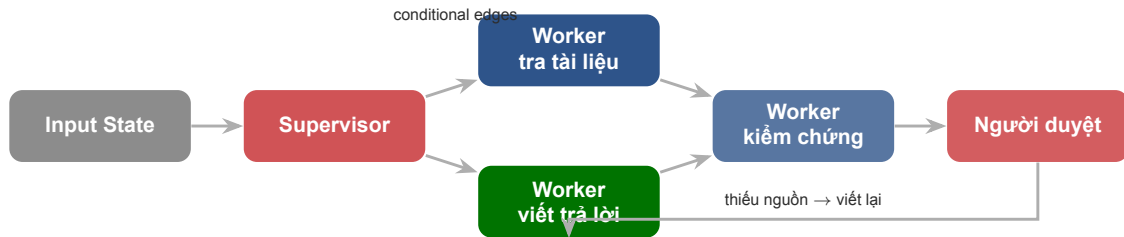
Ranh giới “mỗi node chỉ ghi field của mình” là thứ giúp bạn truy được lỗi phát sinh ở đâu — state không bị ghi đè lẫn lộn.

```
# reads need_verify / need_retrieval from Day09State
def route_to_worker(state: Day09State) -> str:
    if state["need_verify"]:
        return "verify_worker"
    if state["need_retrieval"]:
        return "retrieval_worker"
    return "synthesis_worker"

# routing is EXPLICIT code, not buried in a prompt
builder.add_conditional_edges(
    "supervisor", route_to_worker,
    {w: w for w in ["verify_worker", "retrieval_worker",
                    "synthesis_worker"]})
```

## Một schema

Hàm route đọc thẳng `need_tool/need_retrieval` trên cùng `Day09State` — không có state thứ hai. Routing nằm ở code tường minh nên test được và đọc được, thay vì chôn trong prompt



## Điểm cần nhớ

LangGraph làm rõ **route quyết định ở đâu** (supervisor), **state đi qua đâu** (từng edge), và **người duyệt chen vào chỗ nào** (trước node nhạy cảm).



1. Khi Một Agent Không Còn Đủ
2. Multi-Agent Patterns
3. Supervisor-Worker Pattern
4. MCP - Model Context Protocol
5. A2A - Agent to Agent Communication
6. Orchestration Với LangGraph
- 7. Trace & Độ Tin Cậy**

# Trace & Độ Tin Cậy

---

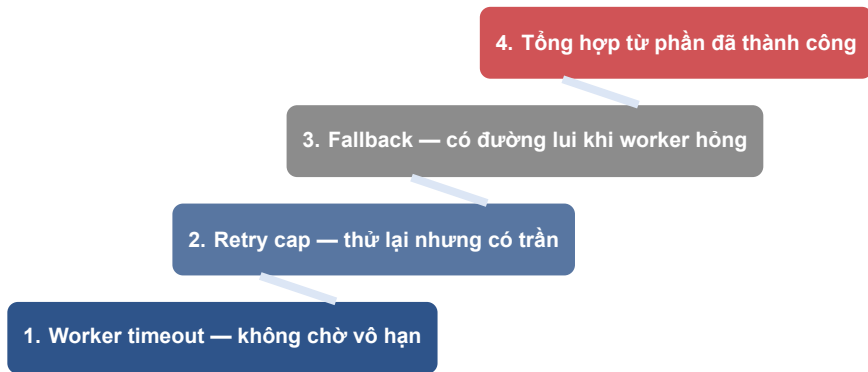
Hệ nhiều agent trả lời sai thì phải biết *khâu nào* sai — trace là thứ cho biết điều đó, và thiết kế hòng có kiểm soát là thứ giữ hệ chạy tiếp

07

```
[
  {
    "timestamp": "11:40:02",
    "agent_id": "supervisor",
    "action": "route",
    "input_summary": "nghỉ thai sản được bao lâu?",
    "output_summary": "route=retrieval_worker (lý do: câu hỏi chính sách)",
    "status": "ok",
    "latency_ms": 180
  },
  {
    "timestamp": "11:40:03",
    "agent_id": "retrieval_worker",
    "action": "call",
    "input_summary": "query=nghỉ thai sản, employee_type=chính thức",
    "output_summary": "chunk=hr_policy_2025.pdf#p12",
    "status": "ok",
    "latency_ms": 420
  },
  {
    "timestamp": "11:40:05",
    "agent_id": "synthesis_worker",
    "action": "synthesize",
    "...": "(other fields as above)",
    "status": "ok",
    "latency_ms": 260
  }
]
```

## Độc kỹ

7 field: timestamp · agent\_id · action · input\_summary · output\_summary · status · latency\_ms. input\_summary là **tóm tắt**, không phải toàn bộ context. Mọi latency\_ms ở đây là số (minh họa).



## Graceful degradation

Với ticket P1 mà một worker timeout, hệ thống không im lặng hỏng: nó trả phần đã chắc chắn, nói rõ phần còn thiếu, và **báo khi không đủ tự tin** thay vì đoán bừa.

| Ngộ nhận                                              | Thực tế                                                                                     |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Nhiều agent = hệ thống tốt hơn                        | Nhiều agent = nhiều phức tạp; chỉ chia vai khi thật cần                                     |
| Supervisor phải là model to nhất<br>MCP và A2A là một | Supervisor chỉ cần đủ để route đúng<br>MCP nối agent với tool; A2A giao việc cho agent khác |
| Multi-agent tự giải quyết vấn đề context              | Vẫn phải quản context cẩn thận ở từng bước                                                  |
| LangGraph chỉ chạy với LangChain                      | LangGraph chạy được với nhiều framework khác                                                |

*Nhớ ranh giới, đừng nhớ khẩu hiệu: nhiều agent không tự nó là tốt hơn*

1

**Multi-agent** là chia vai trò để hệ thống đỡ quá tải và dễ kiểm soát, **không phải** thêm agent cho “ngẫu”.

2

**Supervisor-worker** là điểm khởi đầu thực dụng: route rõ, worker rõ, nối thẳng được với artifact Day 08.

3

**MCP** nối agent với năng lực bên ngoài theo chuẩn chung; **A2A** giao việc giữa các agent bằng contract rõ ràng.

4

**LangGraph** biến orchestration thành graph có state, routing và human-in-the-loop, nên dễ debug và dễ mở rộng.

1. **Glossary:** **supervisor** (agent điều phối, chỉ route) · **worker** (agent chuyên một vai) · **MCP** (chuẩn nổi năng lực ngoài) · **A2A** (giao thức agent giao việc cho agent) · **AgentCard** (JSON mô tả skills/auth/endpoint) · **checkpointer** (lưu state theo thread để resume) · **trace** (nhật ký đường đi của quyết định).
2. LangGraph Quickstart — *docs.langchain.com*, xây supervisor-worker, graph routing, state.
3. MCP Introduction — *modelcontextprotocol.io*, tools / resources / prompts.
4. Liu et al. (2024), *Lost in the Middle: How Language Models Use Long Contexts*, TACL.
5. Sumers, Yao, Narasimhan, Griffiths (2023), *Cognitive Architectures for Language Agents (CoALA)*.
6. Anthropic (19/12/2024), *Building Effective Agents* — orchestrator-workers và các pattern khác.

# Hỏi & Đáp

---

*Một agent rất giỏi có thể làm nhiều việc. Nhưng hệ thống tốt hơn thường bắt đầu từ câu hỏi: nên chia vai trò ở đâu?*